

# Pillar 5: Capstone Project

Submitted by: **Darwin A. Nepomuceno**

**Post Graduate Diploma in Artificial Intelligence and Machine Learning**

---



## CTG — Package Usage Guide

The `src/` folder is a **modular, importable version** of the full pipeline. Instead of running cells in order across 8 notebooks, you can call individual functions directly in any script, notebook, or API.

---

### Project Structure

```
ctg_src/
├── src/
│   ├── __init__.py    ← package entry point
│   └── data_loader.py ← load raw Excel + Arrow-safe dtype
├── fix
│   ├── preprocessor.py ← deduplicate, map target, winsorize
│   └── pipeline.py     ← split → scale → resample (no-leakage
order)
├── model.py           ← cost function, training, threshold
sweep, Optuna
├── tests/
│   └── test_src.py    ← unit tests for every module
```

---

### Setup

```
cd ctg_src
pip install pandas numpy scikit-learn imbalanced-learn xgboost optuna
openpyxl
```

No `pip install -e .` needed — just make sure you run Python from the `ctg_src/` directory (or add it to your `PYTHONPATH`).

---

### Module 1 — `data_loader`

Load the raw Excel file and strip Arrow/PyArrow dtype issues automatically.

```
from src.data_loader import load_raw, describe_raw
```

```

# Load and fix dtypes
df = load_raw("test.xlsx")

# Quick summary
info = describe_raw(df)
print(info)
# {'shape': (484, 23), 'n_missing': 0, 'n_duplicates': 6, ...}

```

---

## Module 2 — preprocessor

Deduplicate, map NSP → binary target, winsorize outliers.

```

from src.preprocessor import preprocess, remove_duplicates, map_target,
winsorize

```

```

# Full pipeline in one call
df_clean, features = preprocess(df)

print(f"Clean shape : {df_clean.shape}")
print(f"Features      : {features}")
print(f"Target dist   : {df_clean['target'].value_counts().to_dict()}")
Or call individual steps:

```

```

# Step by step
df, n_removed = remove_duplicates(df)
print(f"Removed {n_removed} duplicates")

df = map_target(df)          # adds 'target' column (NSP 1→0, 2&3→1)

from src.preprocessor import get_feature_names
features = get_feature_names(df)

df = winsorize(df, features) # caps extremes at 1st/99th pct + physio
bounds

```

---

## Module 3 — pipeline

Split → Scale → Resample in the correct no-leakage order.

```

from src.pipeline import build_pipeline

X = df_clean[features]
y = df_clean["target"]

# Full pipeline – returns a dict of all train/test variants
data = build_pipeline(X, y)

# Unpack what you need
X_train_scaled = data["X_train_scaled"] # scaled, no resampling
X_test_scaled  = data["X_test_scaled"]  # NEVER resampled
y_train        = data["y_train"]

```

```

y_test          = data["y_test"]
X_train_smote   = data["X_train_smote"]    # SMOTE oversampled
y_train_smote   = data["y_train_smote"]
scaler          = data["scaler"]           # fitted StandardScaler –
save this!

```

Or use individual functions:

```

from src.pipeline import split, scale, resample_smote,
compute_class_weights

X_train, X_test, y_train, y_test = split(X, y)

# Scale – fit on train only, transform both
X_train_scaled, X_test_scaled, scaler = scale(X_train, X_test)

# Resample – training set only
X_train_smote, y_train_smote = resample_smote(X_train_scaled, y_train)

# Class weights (alternative to resampling)
cw = compute_class_weights(y_train)
print(cw)  # {0: 0.88, 1: 1.16}

```

---

## Module 4 — model

Train, evaluate, sweep thresholds, tune with Optuna.

### Evaluate a model

```

from src.model import evaluate, build_best_model
import numpy as np

scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()
model = build_best_model(scale_pos_weight)

results = evaluate(
    model,
    X_train_scaled, y_train,
    X_test_scaled, y_test,
    threshold=0.10,
    name="XGB (scale_pos_weight)",
)
print(results)
# {'Model': 'XGB (scale_pos_weight)', 'Accuracy': 0.9271,
#  'Recall': 0.9286, 'Cost': 34, ...}

```

### Compute clinical cost directly

```

from src.model import compute_cost

cost = compute_cost(y_test, y_pred)
print(f"Cost = {cost}")  # FP×1 + FN×10

```

## Sweep thresholds

```
from src.model import sweep_thresholds, select_optimal_threshold

model.fit(X_train_scaled, y_train)
df_thresh = sweep_thresholds(model, X_test_scaled, y_test)

print(df_thresh.head(10))
#   Threshold  Cost  Recall  Precision    F1  FP  FN
# 0      0.10   39   0.929    0.813  0.867   9   3
# ...

optimal_t = select_optimal_threshold(df_thresh)
print(f"Optimal threshold: {optimal_t}") # 0.10
```

## Tune with Optuna

```
from src.model import tune_xgboost

best_params, study = tune_xgboost(
    X_train_scaled, y_train,
    n_trials=50,
    cv_folds=5,
)
print("Best params:", best_params)
print("Best CV cost:", -study.best_value) # negate back to positive cost
```

---

## Full Pipeline in One Script

```
from src.data_loader import load_raw
from src.preprocessor import preprocess
from src.pipeline import build_pipeline
from src.model import build_best_model, evaluate,
sweep_thresholds, select_optimal_threshold

# 1. Load
df = load_raw("test.xlsx")

# 2. Clean
df_clean, features = preprocess(df)
X = df_clean[features]
y = df_clean["target"]

# 3. Split → Scale → Resample
data = build_pipeline(X, y)

# 4. Train best model
spw = (data["y_train"] == 0).sum() / (data["y_train"] == 1).sum()
model = build_best_model(spw)

# 5. Find optimal threshold
model.fit(data["X_train_scaled"], data["y_train"])
df_thresh = sweep_thresholds(model, data["X_test_scaled"],
```

```

data["y_test"])
threshold = select_optimal_threshold(df_thresh)
print(f"Optimal threshold: {threshold}")

# 6. Final evaluation
results = evaluate(
    model,
    data["X_train_scaled"], data["y_train"],
    data["X_test_scaled"], data["y_test"],
    threshold=threshold,
    name="XGB (scale_pos_weight)",
)
for k, v in results.items():
    print(f" {k:<12}: {v}")

```

Expected output:

```

Optimal threshold: 0.10
Model          : XGB (scale_pos_weight)
Accuracy       : 0.9271
Precision      : 0.907
Recall         : 0.9286
F1             : 0.9176
Cost           : 34
ROC_AUC        : 0.9749
AP             : 0.9693
TP             : 39
FP            : 4
TN            : 50
FN            : 3

```

## What each module is responsible for

| Module       | Responsibility                     | Key rule                                 |
|--------------|------------------------------------|------------------------------------------|
| data_loader  | Load Excel → strip Arrow dtypes    | Never modifies values                    |
| preprocessor | Deduplicate, map target, winsorize | No statistical fitting                   |
| pipeline     | Split → Scale → Resample           | Split BEFORE scale; resample AFTER split |
| model        | Train, evaluate, tune              | Test set used exactly once               |

## Running the unit tests

```

cd ctg_src
pytest tests/ -v

```

Tests cover every public function in all four modules.

## Saving and reusing the scaler

The `scaler` returned by `pipeline.build_pipeline()` or `pipeline.scale()` must be saved alongside the model for production use:

```
import pickle
```

```
# Save
```

```
with open("deployment_artifact.pkl", "wb") as f:
    pickle.dump({"model": model, "scaler": scaler, "features":
features}, f)
```

```
# Load and use for new patient data
```

```
with open("deployment_artifact.pkl", "rb") as f:
    artifact = pickle.load(f)
```

```
import pandas as pd
```

```
new_patient = pd.DataFrame([{"LB": 120, "ASTV": 40, ...}])
```

```
X_new_scaled =
```

```
artifact["scaler"].transform(new_patient[artifact["features"]])
```

```
proba = artifact["model"].predict_proba(X_new_scaled)[0, 1]
```

```
print("Risk probability:", proba)
```

**Important:** always use the saved scaler — never refit it on new data. Refitting would apply different mean/std values than the model was trained on, producing incorrect predictions.